1. Saúde reprodutiva e planejamento familiar das mulheres

2. Membros

Antônio Alberto Pereira de Oliveira - 2021086504

Kauan Valente - 2023060464

Maria Eduarda Sampaio - 2022055831

3. Descrição dos dados

URL https://www.kaggle.com/datasets/konradb/wellbeing-of-women-in-52-countries

Para reproduzir: baixar o dataset no Kaggle e colocar os arquivos `livwell.csv` e `indicators.csv` em `data/raw/`.

Domínio Saúde reprodutiva e planejamento familiar das mulheres e sua relação com escolaridade, trabalho, autonomia nas decisões e violência doméstica. As consultas concentram-se em três áreas temáticas: Saúde reprodutiva, Educação e trabalho, e Autonomia e violência. Os demais indicadores selecionados (demografia, domicílio e condições de vida, saúde infantil e nutrição) permanecem no banco como contexto para análises futuras. Os indicadores vêm de pesquisas DHS (*Demographic and Health Surveys*) e permitem comparar países, regiões e períodos.

Processamento

O conjunto de dados LivWell é composto originalmente por 1.832 registros e 409 colunas (`livwell.csv`), além de um dicionário com 265 indicadores (`indicators.csv`). Cada registro corresponde a uma região de um país em uma pesquisa DHS, e cada indicador ocupa uma coluna (com uma coluna `_se` adicional para o erro-padrão, quando disponível).

O processamento, implementado nas células abaixo, segue as etapas:

1. **Seleção de indicadores** — foram selecionados manualmente 100 indicadores relacionados ao domínio do trabalho, distribuídos em 16 categorias. A seleção está em `data/processed/filtered_indicators.csv`.
2. **Filtragem de `livwell.csv`** — foram mantidas apenas as colunas de identificação (país, região, pesquisa, data da entrevista) e as colunas de valor e erro-padrão dos 100 indicadores. Nenhum registro foi removido.

3. **Normalização** — os dados foram decompostos em sete relações: `Pais`, `Regiao`, `Pesquisa`, `ColetaPesquisaRegiao`, `CategoriaIndicador`, `Indicador` e `Medicao`. As colunas de indicadores foram convertidas em linhas (uma medição por pesquisa × região × indicador), removendo apenas as medições sem valor observado e mantendo erros-padrão ausentes como nulos.
4. **Carga e verificação** — as relações foram inseridas em um banco SQLite com chaves primárias, estrangeiras e restrições de unicidade. A validação de chaves estrangeiras é desativada durante a carga (`PRAGMA foreign_keys = 0`) e a integridade referencial é verificada ao final.

Ajustes nos dados originais

- **País**: extraído como entidade própria. Além do nome, recebeu o continente, atribuído pelo grupo (o dataset não possui essa informação). Turquia e Armênia, países transcontinentais, foram classificados na Ásia.
- **Data da entrevista**: `interview_year_mean`, `interview_month_mean` e `CMC_interview_mean` variam entre as regiões de uma mesma pesquisa. Por isso pertencem ao relacionamento `ColetaPesquisaRegiao` (pesquisa × região), e não à pesquisa. Em `Pesquisa`, `CMC_interview_mean` é mantido como atributo derivado (média das regiões).
- **Categorias**: no dataset original a categoria é apenas um texto repetido em cada indicador. Ela foi extraída para a entidade `CategoriaIndicador`, enriquecida com o nome em português e uma área temática (agrupamento de categorias definido pelo grupo).
- **Tipo de indicador**: cada indicador recebeu um tipo (percentual, média, taxa, índice ou valor monetário), deduzido do sufixo do código e da descrição.
- **Nomes de região**: duas regiões aparecem com grafias diferentes entre pesquisas (`Hauncavelica` / `Huancavelica` no Peru e `Western  Cape` / `Western Cape` na África do Sul). Foi mantida a grafia da pesquisa mais recente.
- **Códigos de indicador**: alguns códigos usam hífen (`FF_ASFR_15-19`) enquanto a coluna correspondente no CSV usa ponto (`FF_ASFR_15.19`); o mapeamento é feito na filtragem.

3.1 Configuração

In [15]:
```
import sqlite3
from pathlib import Path

import pandas as pd

RAW = Path("data/raw")
PROCESSED = Path("data/processed")
DB_PATH = Path("saudefeminina.db")

# Fecha a conexão de uma execução anterior (no Windows, o arquivo aberto não pode s
```

```
if "con" in globals():
    con.close()

# O banco é sempre recriado do zero a partir dos CSVs
DB_PATH.unlink(missing_ok=True)
con = sqlite3.connect(DB_PATH)
cur = con.cursor()
```

3.2 Leitura dos dados brutos

In [16]:
```
livwell = pd.read_csv(RAW / "livwell.csv")
indicadores_raw = pd.read_csv(RAW / "indicators.csv")
selecao = pd.read_csv(PROCESSED / "filtered_indicators.csv")

print("livwell.csv:", livwell.shape)
print("indicators.csv:", indicadores_raw.shape)
print("Indicadores selecionados:", len(selecao))

# Todo indicador selecionado deve existir no dicionário original
assert selecao["indicator_code"].isin(indicadores_raw["indicator_code"]).all()

selecao["indicator_category"].value_counts().to_frame("indicadores")
```

```
livwell.csv: (1832, 409)
indicators.csv: (265, 3)
Indicadores selecionados: 100
```

Out[16]:

indicator_category	indicadores
Individual demographic information	13
Energy and information	12
Reproductive health and fertility	11
Decision power	10
Domestic Violence	10
Education	9
Fertility – complex indicators	9
Health – Birth level	5
Household characteristics – Womens level	4
Work status	4
Fertility preferences	3
Household Wealth	3
Energy and information – per urban/rural area	2
Household demographics – Household level	2
Socio-economic indicators	2
Nutrition	1

3.3 Filtragem das colunas

In [17]:
```
COLUNAS_ID = [
    "country_name", "country_code", "year",
    "region_num_harmonized", "region_name_harmonized",
    "SurveyId", "interview_year_mean", "interview_month_mean", "CMC_interview_mean"
]

# Código do indicador -> coluna de valor e coluna de erro-padrão (se existir)
mapa = {}
for codigo in selecao["indicator_code"]:
    coluna = codigo.replace("-", ".")
    assert coluna in livwell.columns, f"Coluna não encontrada: {coluna}"
    coluna_se = coluna + "_se"
    mapa[codigo] = (coluna, coluna_se if coluna_se in livwell.columns else None)

colunas = COLUNAS_ID + [c for par in mapa.values() for c in par if c]
filtrado = livwell[colunas].copy()

print("Dados filtrados:", filtrado.shape)
print("Indicadores sem coluna de erro-padrão:", sum(se is None for _, se in mapa.va
```

```
Dados filtrados: (1832, 192)
Indicadores sem coluna de erro-padrão: 17
```

3.4 Criação do esquema

In [18]:

```
cur.executescript("""
PRAGMA foreign_keys = 0;

CREATE TABLE Pais (
    country_code TEXT PRIMARY KEY,
    country_name TEXT NOT NULL,
    continent    TEXT NOT NULL
);

CREATE TABLE Regiao (
    country_code           TEXT    NOT NULL REFERENCES Pais(country_code),
    region_num_harmonized  INTEGER NOT NULL,
    region_name_harmonized TEXT    NOT NULL,
    PRIMARY KEY (country_code, region_num_harmonized)
);

CREATE TABLE Pesquisa (
    SurveyId           TEXT PRIMARY KEY,
    country_code       TEXT    NOT NULL REFERENCES Pais(country_code),
    year               INTEGER NOT NULL,
    CMC_interview_mean REAL,
    UNIQUE (SurveyId, country_code)
);

CREATE TABLE ColetaPesquisaRegiao (
    SurveyId              TEXT    NOT NULL,
    country_code          TEXT    NOT NULL,
    region_num_harmonized INTEGER NOT NULL,
    interview_year_mean   REAL,
    interview_month_mean  REAL,
    CMC_interview_mean    REAL,
    PRIMARY KEY (SurveyId, country_code, region_num_harmonized),
    FOREIGN KEY (SurveyId, country_code)
        REFERENCES Pesquisa(SurveyId, country_code),
    FOREIGN KEY (country_code, region_num_harmonized)
        REFERENCES Regiao(country_code, region_num_harmonized)
);

CREATE TABLE CategoriaIndicador (
    category_id      INTEGER PRIMARY KEY,
    category_name    TEXT NOT NULL UNIQUE,
    category_name_pt TEXT NOT NULL,
    category_area    TEXT NOT NULL
);

CREATE TABLE Indicador (
    indicator_code        TEXT PRIMARY KEY,
    indicator_description TEXT NOT NULL,
    indicator_type        TEXT NOT NULL,
    category_id           INTEGER NOT NULL REFERENCES CategoriaIndicador(category_i
```

```
);

CREATE TABLE Medicao (
    measurement_id        INTEGER PRIMARY KEY,
    SurveyId              TEXT    NOT NULL REFERENCES Pesquisa(SurveyId),
    country_code          TEXT    NOT NULL,
    region_num_harmonized INTEGER NOT NULL,
    indicator_code        TEXT    NOT NULL REFERENCES Indicador(indicator_code),
    value                 REAL    NOT NULL,
    standard_error        REAL,
    FOREIGN KEY (country_code, region_num_harmonized)
        REFERENCES Regiao(country_code, region_num_harmonized),
    UNIQUE (SurveyId, country_code, region_num_harmonized, indicator_code)
);
""")

con.commit()
```

3.5 Inserção dos dados

Pais, Regiao, Pesquisa e ColetaPesquisaRegiao

In [19]:

```
CONTINENTES = {
    "África": ["BDI", "BEN", "BFA", "CIV", "CMR", "COD", "EGY", "ETH", "GAB", "GHA"
               "KEN", "LBR", "LSO", "MAR", "MDG", "MLI", "MOZ", "MWI", "NAM", "NER"
               "RWA", "SEN", "SLE", "TGO", "TZA", "UGA", "ZAF", "ZMB", "ZWE"],
    "América": ["BOL", "COL", "GTM", "HND", "HTI", "NIC", "PER"],
    "Ásia": ["ARM", "BGD", "IDN", "IND", "JOR", "KHM", "MDV", "NPL", "PAK", "PHL",
             "TLS", "TUR", "VNM"],
}
continente_de = {pais: cont for cont, paises in CONTINENTES.items() for pais in pai

pais = filtrado[["country_code", "country_name"]].drop_duplicates("country_code")
pais["continent"] = pais["country_code"].map(continente_de)
assert pais["continent"].notna().all()
pais = pais.sort_values("country_code")
pais.to_sql("Pais", con, if_exists="append", index=False)

regiao = (
    filtrado.sort_values("year", ascending=False)
    .drop_duplicates(["country_code", "region_num_harmonized"])
    [["country_code", "region_num_harmonized", "region_name_harmonized"]]
)
regiao.to_sql("Regiao", con, if_exists="append", index=False)

assert (filtrado.groupby("SurveyId")[["country_code", "year"]].nunique() == 1).all(
pesquisa = filtrado.groupby("SurveyId", as_index=False).agg(
    country_code=("country_code", "first"),
    year=("year", "first"),
    CMC_interview_mean=("CMC_interview_mean", "mean"),
)
pesquisa.to_sql("Pesquisa", con, if_exists="append", index=False)

coleta = filtrado[[
```

```
    "SurveyId", "country_code", "region_num_harmonized",
    "interview_year_mean", "interview_month_mean", "CMC_interview_mean",
]]
coleta.to_sql("ColetaPesquisaRegiao", con, if_exists="append", index=False)

print("Países:", len(pais), "| Regiões:", len(regiao),
      "| Pesquisas:", len(pesquisa), "| Pares pesquisa x região:", len(coleta))
```

```
Países: 52 | Regiões: 481 | Pesquisas: 199 | Pares pesquisa x região: 1832
```

CategoriaIndicador e Indicador

In [20]:

```
CATEGORIAS = {
    "Decision power": ("Poder de decisão", "Autonomia e violência"),
    "Domestic Violence": ("Violência doméstica", "Autonomia e violência"),
    "Education": ("Educação", "Educação e trabalho"),
    "Work status": ("Situação de trabalho", "Educação e trabalho"),
    "Energy and information": ("Energia e informação", "Domicílio e condições de vi
    "Energy and information – per urban/rural area": ("Energia e informação – por á
    "Household characteristics – Womens level": ("Características do domicílio – ní
    "Household demographics – Household level": ("Demografia do domicílio – nível d
    "Household Wealth": ("Riqueza do domicílio", "Domicílio e condições de vida"),
    "Socio-economic indicators": ("Indicadores socioeconômicos", "Domicílio e condi
    "Individual demographic information": ("Informações demográficas individuais",
    "Fertility – complex indicators": ("Fecundidade – indicadores compostos", "Saúd
    "Fertility preferences": ("Preferências de fecundidade", "Saúde reprodutiva"),
    "Reproductive health and fertility": ("Saúde reprodutiva e fecundidade", "Saúde
    "Health – Birth level": ("Saúde – nível do nascimento", "Saúde infantil e nutri
    "Nutrition": ("Nutrição", "Saúde infantil e nutrição"),
}
assert set(selecao["indicator_category"]) == set(CATEGORIAS)

categoria = pd.DataFrame(
    [(nome, pt, area) for nome, (pt, area) in CATEGORIAS.items()],
    columns=["category_name", "category_name_pt", "category_area"],
)
categoria.insert(0, "category_id", range(1, len(categoria) + 1))
categoria.to_sql("CategoriaIndicador", con, if_exists="append", index=False)


def tipo_indicador(codigo):
    """Tipo de medida, deduzido do sufixo do código e da descrição do indicador."""
    if codigo.endswith("_p"):
        return "Percentual"
    if codigo.endswith("_mean"):
        return "Média"
    if codigo.startswith(("FF_", "HL_")):
        return "Taxa"
    if codigo == "gdp_pc":
        return "Valor monetário"
    return "Índice"


indicador = selecao.merge(categoria, left_on="indicator_category", right_on="catego
indicador["indicator_type"] = indicador["indicator_code"].map(tipo_indicador)
```

```
indicador = indicador[["indicator_code", "indicator_description", "indicator_type",
indicador.to_sql("Indicador", con, if_exists="append", index=False)

print("Categorias:", len(categoria), "| Indicadores:", len(indicador))
indicador["indicator_type"].value_counts().to_frame("indicadores")
```

```
Categorias: 16 | Indicadores: 100
```

Out[20]:

indicator_type	indicadores
Percentual	67
Média	15
Taxa	14
Índice	3
Valor monetário	1

Medicao

Cada coluna de indicador vira um conjunto de linhas (formato largo → longo).

In [21]:

```
chaves = ["SurveyId", "country_code", "region_num_harmonized"]
partes = []
for codigo, (coluna, coluna_se) in mapa.items():
    parte = filtrado[chaves].copy()
    parte["indicator_code"] = codigo
    parte["value"] = filtrado[coluna]
    parte["standard_error"] = filtrado[coluna_se] if coluna_se else None
    partes.append(parte)

medicao = pd.concat(partes, ignore_index=True).dropna(subset=["value"])
medicao.to_sql("Medicao", con, if_exists="append", index=False)
con.commit()

print("Medições:", len(medicao))
```

```
Medições: 147829
```

3.6 Criação de regras de exclusão

In [22]:

```
# Ativa integridade referencial nesta conexão
cur.execute("PRAGMA foreign_keys = ON")

# Gatilhos para impedir exclusões de entidades compartilhadas e que ainda possuem r
restricoes = {
    "Pais": [
        ("Regiao", "country_code = OLD.country_code"),
        ("Pesquisa", "country_code = OLD.country_code"),
    ],
    "Regiao": [
```

```
        (
            "ColetaPesquisaRegiao",
            """country_code = OLD.country_code
               AND region_num_harmonized = OLD.region_num_harmonized"""
        ),
        (
            "Medicao",
            """country_code = OLD.country_code
               AND region_num_harmonized = OLD.region_num_harmonized"""
        ),
    ],
    "CategoriaIndicador": [
        ("Indicador", "category_id = OLD.category_id"),
    ],
    "Indicador": [
        ("Medicao", "indicator_code = OLD.indicator_code"),
    ],
}

for tabela_pai, dependencias in restricoes.items():
    for tabela_filha, condicao in dependencias:
        nome = f"trg_restrict_{tabela_pai}_{tabela_filha}"

        cur.execute(f"""
            CREATE TRIGGER IF NOT EXISTS {nome}
            BEFORE DELETE ON {tabela_pai}
            FOR EACH ROW
            WHEN EXISTS (
                SELECT 1
                FROM {tabela_filha}
                WHERE {condicao}
            )
            BEGIN
                SELECT RAISE(
                    ABORT,
                    'Exclusao bloqueada: existem registros dependentes em {tabela_f
                );
            END;
        """)

# CASCADE: excluir as coletas regionais de uma pesquisa quando a própria pesquisa f
cur.execute("""
    CREATE TRIGGER IF NOT EXISTS trg_cascade_Pesquisa_Coleta
    BEFORE DELETE ON Pesquisa
    FOR EACH ROW
    BEGIN
        DELETE FROM ColetaPesquisaRegiao
        WHERE SurveyId = OLD.SurveyId
          AND country_code = OLD.country_code;
    END;
""")

# CASCADE: excluir as medições de uma pesquisa quando a própria pesquisa for excluí
cur.execute("""
    CREATE TRIGGER IF NOT EXISTS trg_cascade_Pesquisa_Medicao
    BEFORE DELETE ON Pesquisa
```

```
    FOR EACH ROW
    BEGIN
        DELETE FROM Medicao
        WHERE SurveyId = OLD.SurveyId;
    END;
""")

con.commit()

# Lista os gatilhos instalados
gatilhos = cur.execute("""
    SELECT name, tbl_name, sql
    FROM sqlite_master
    WHERE type = 'trigger'
    ORDER BY name
""").fetchall()

for nome, tabela, definicao in gatilhos:
    print(f"\n{nome} — tabela: {tabela}")
```

```
trg_cascade_Pesquisa_Coleta — tabela: Pesquisa

trg_cascade_Pesquisa_Medicao — tabela: Pesquisa

trg_restrict_CategoriaIndicador_Indicador — tabela: CategoriaIndicador

trg_restrict_Indicador_Medicao — tabela: Indicador

trg_restrict_Pais_Pesquisa — tabela: Pais

trg_restrict_Pais_Regiao — tabela: Pais

trg_restrict_Regiao_ColetaPesquisaRegiao — tabela: Regiao

trg_restrict_Regiao_Medicao — tabela: Regiao
```

3.7 Verificação

In [23]:

```
cur.execute("PRAGMA foreign_keys = 1")
erros_fk = cur.execute("PRAGMA foreign_key_check").fetchall()
print("Erros de integridade referencial:", erros_fk)
assert not erros_fk

orfas = cur.execute("""
    SELECT COUNT(*) FROM Medicao m
    LEFT JOIN ColetaPesquisaRegiao a USING (SurveyId, country_code, region_num_harm
    WHERE a.SurveyId IS NULL
""").fetchone()[0]
print("Medições sem par em ColetaPesquisaRegiao:", orfas)
assert orfas == 0

tabelas = ["Pais", "Regiao", "Pesquisa", "ColetaPesquisaRegiao", "CategoriaIndicado
pd.DataFrame(
    {"registros": [cur.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0] for t in
```

```
    index=tabelas,
)
```

```
Erros de integridade referencial: []
Medições sem par em ColetaPesquisaRegiao: 0
```

Out[23]:

	registros
Pais	52
Regiao	481
Pesquisa	199
ColetaPesquisaRegiao	1832
CategoriaIndicador	16
Indicador	100
Medicao	147829

In [24]:

```
# Amostra do banco normalizado
pd.read_sql_query("""
    SELECT pa.country_name, pa.continent, r.region_name_harmonized,
           p.SurveyId, p.year, a.interview_month_mean,
           c.category_name_pt, i.indicator_code, i.indicator_type,
           m.value, m.standard_error
    FROM Medicao m
    JOIN ColetaPesquisaRegiao a   USING (SurveyId, country_code, region_num_harmoni
    JOIN Regiao r    USING (country_code, region_num_harmonized)
    JOIN Pais pa     USING (country_code)
    JOIN Pesquisa p  USING (SurveyId)
    JOIN Indicador i USING (indicator_code)
    JOIN CategoriaIndicador c USING (category_id)
    WHERE i.indicator_code = 'RH_contr_modern_p'
    LIMIT 5
""", con)
```

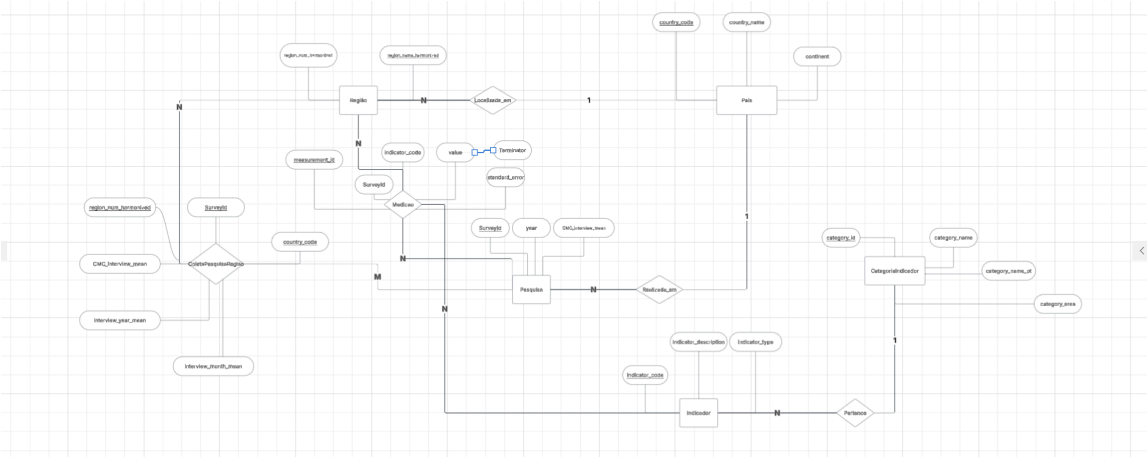
Out[24]:

	country_name	continent	region_name_harmonized	SurveyId	year	interview_month
0	Armenia	Ásia	Aragatsotn	AM2000DHS	2000	
1	Armenia	Ásia	Ararat	AM2000DHS	2000	
2	Armenia	Ásia	Armavir	AM2000DHS	2000	
3	Armenia	Ásia	Gegharkunik	AM2000DHS	2000	
4	Armenia	Ásia	Lori	AM2000DHS	2000	

4. Diagrama ER

Para ajudar no entendimento das relações do sistema desenvolvido, foi construído um diagrama que pode ser acessado no link abaixo.

https://lucid.app/lucidchart/74952537-0644-48d7-9527-9f8bb02de887/edit?viewport_loc=-2663%2C-1030%2C5760%2C2562%2C0_0&invitationId=inv_f3bf9b73-e139-47f8-99a3-6d149bb52dd9



5. Diagrama relacional

O diagrama relacional mostra as sete tabelas do banco, com suas chaves primárias (PK), chaves estrangeiras (FK) e as ligações entre elas:

